Modelagem de sistema *e-commerce*

Eduardo Alves Calza

[1]Centro de Ciências Tecnológicas - Universidade Estadual do Norte do Paraná (UENP)
Bandeirantes - PR - Brasil

eduardo.calza@discente.uenp.edu.br

1. Introdução

Este trabalho tem como objetivo modelar um software de *e-commerce* para uma pequena empresa chamada Geração Livre. A empresa é uma loja especializada na venda de tecidos e itens para artesanato e costura.

Durante a conversa com a proprietária, foi identificada uma das principais dificuldades enfrentadas pelo negócio: a limitação geográfica das vendas, restritas à cidade onde a loja está localizada. A empresária expressou o desejo de expandir seu alcance por meio das vendas online, permitindo que clientes de outras regiões também possam adquirir seus produtos.

Muitos consumidores relataram a escassez de lojas semelhantes em suas cidades, o que os leva a se deslocar até Bandeirantes apenas para comprar na Geração Livre. Dessa forma, a criação de uma plataforma de vendas online representa uma excelente oportunidade para ampliar o alcance do negócio e potencializar as vendas.

2. Metodologia e Método

A metodologia deste trabalho adotará uma abordagem estruturada para a modelagem de requisitos de software, partindo de uma técnica ágil focada no usuário para, em seguida, detalhá-la em uma especificação mais tradicional. O processo será dividido em etapas principais, fundamentadas na literatura de Engenharia de Software, com destaque para Menolli (2024).

Inicialmente, será empregado um método exploratório com uma abordagem qualitativa. O objetivo desta fase é investigar o problema e compreender em profundidade o contexto da empresa ”Geração Livre”e suas necessidades. Para isso, serão utilizadas técnicas de coleta de dados como entrevistas com a proprietária e a elaboração de questionários para potenciais usuários. Os dados coletados servirão de base para a definição das hipóteses do projeto.

Concluída a fase exploratória, o trabalho avançará para um método descritivo, focado em detalhar sistematicamente as características e funcionalidades do software. Esta etapa começará com a engenharia de requisitos, utilizando duas técnicas principais:

- Histórias de Usuário: Para descrever as funcionalidades sob o ponto de vista do usuário, focando no valor agregado.
- Casos de Uso: Para detalhar tecnicamente o comportamento do sistema e as interações com os atores, com base nas histórias de usuário.

Na sequência do método descritivo, será realizada a modelagem estrutural do sistema. Esta fase incluirá o desenvolvimento do modelo de classes e do diagrama de classes,

que irão especificar as entidades do software, seus atributos e os relacionamentos entre elas. Por fim, será definida a arquitetura da aplicação, estabelecendo os componentes de software, suas responsabilidades e como eles interagem para atender aos requisitos levantados.

3. Desenvolvimento

3.1. Histórias de Usuário

As *User Storys* (Histórias de usuário) constituem uma técnica amplamente empregada na engenharia de requisitos, especialmente em métodos ágeis de desenvolvimento de software. Segundo Menolli (2024), as histórias de usuário "descrevem as principais funcionalidades do ponto de vista de um usuário ou cliente de um sistema ou software", permitindo compreender de forma clara o que o usuário deseja e qual valor aquela funcionalidade agrega ao produto final.

Essa técnica possibilita dividir o sistema em pequenas funcionalidades entregáveis, favorecendo o planejamento incremental e a priorização das entregas. Cada história representa uma unidade funcional independente, escrita em linguagem simples e compreensível, que expressa quem é o usuário, o que ele deseja realizar e qual o objetivo ou benefício esperado.

De acordo com o autor, "as histórias são compostas de três aspectos", representados na Figura 1.:

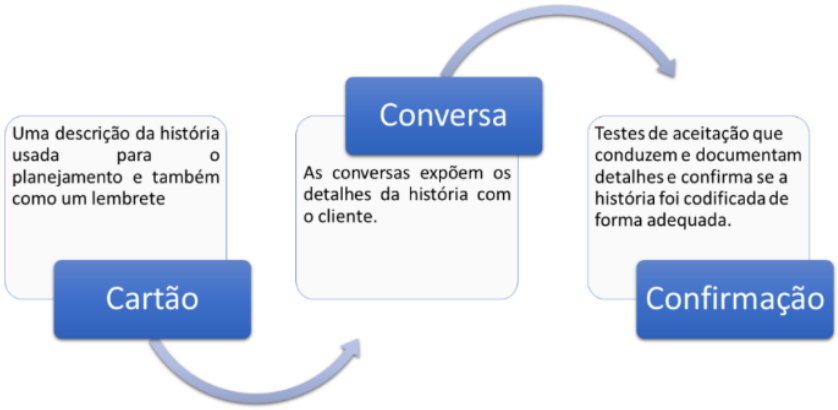


Figura 1. Os três aspectos das histórias de usuário

e para a criação das histórias, foi utilzado a seguinte estratégia:

> ***Como*** *[tipo de usuário],* ***quero*** *[realizar uma ação],* ***para*** *[atingir um objetivo].*
>
> (Menolli, 2024[1])

Além disso, há uma forma de confirmar se a história foi bem implementada, definir testes de aceitação. Toda história deve ser associada ao menos a um teste de aceitação. No entanto, o ideal é ter um conjunto de testes, de forma a garantir que a história seja implementada de acordo com a expectativa do cliente. Estes testes definem as respostas que

a funcionalidade deve fornecer de acordo com a utilização por parte do usuário [(Menolli, 2024[1])].

Outra funcionalidade proporcionada pelas histórias de usuário é a definição de prioridades para as tarefas que devem ser executadas. Para esse fim, existem diversas abordagens de categorização. No presente trabalho, será adotada a técnica MoSCoW, que classifica os requisitos e tarefas segundo quatro categorias: *Must have* (deve ter), *Should have* (deveria ter), *Could have* (poderia ter) e *Won't have* (não terá nesta versão). Após a etapa de priorização, será apresentada uma breve justificativa para cada classificação tomada, visando garantir transparência e alinhamento com os objetivos do projeto.

A aplicação dessa técnica é essencial para garantir o alinhamento contínuo entre cliente e equipe de desenvolvimento, promovendo um fluxo de feedback constante, característica fundamental dos métodos ágeis. Tal abordagem possibilita ajustes rápidos nas funcionalidades e na priorização de requisitos conforme novas demandas surgem durante o projeto.

Neste trabalho, a técnica de Histórias de Usuário foi empregada para modelar um *e-commerce* voltado a uma loja de tecidos e artigos para artesanato. Considerando a diversidade de funcionalidades que um sistema desse tipo envolve, foram selecionadas e descritas as principais histórias de usuário, representando as funcionalidades mais relevantes para o funcionamento do *software*.

A seguir, são apresentadas as *User Stories* desenvolvidas para o projeto.

3.1.1. Cadastro/Login

ID: US01

Como cliente, **quero** criar uma conta e fazer login no site **para** acessar meus dados pessoais, histórico de compras e realizar pedidos de forma segura.

Critérios de Aceitação:

- O sistema deve permitir o cadastro de novos usuários, solicitando nome, e-mail, telefone e senha.
- O sistema deve validar se os campos estão na formatação correta e exibir uma mensagem adequada.
- O sistema deve validar se o e-mail já está cadastrado e exibir uma mensagem adequada.
- O usuário deve conseguir fazer login com e-mail e senha válidos.
- Caso as credenciais estejam incorretas, o sistema deve exibir uma mensagem de erro clara.
- O sistema deve oferecer opção de recuperação de senha por e-mail.
- Após o login, o usuário deve ser redirecionado à página inicial logado (com o nome exibido no canto superior).
- O usuário deve poder sair da conta (*logout*) com segurança.

Critérios de Priorização:

Prioridade: Alta (*Must Have*)
Justificativa: O login e cadastro são essenciais para identificar o usuário, salvar favoritos, histórico e permitir a finalização das compras.

3.1.2. Listagem de produtos

ID: US02

Como cliente, **quero** visualizar todos os produtos disponíveis na loja com suas imagens, nomes e preços, **para** poder escolher o que desejo comprar. .

Critérios de Aceitação:

- O sistema deve exibir uma lista de produtos com imagem, nome e preço.
- Cada produto deve estar organizado em uma grade para facilitar a navegação.
- O usuário deve poder clicar em um produto para ver seus detalhes.
- A página deve permitir paginação ou rolagem contínua se houver muitos produtos.
- O sistema deve carregar os produtos rapidamente, sem travamentos visíveis.
- Caso não haja produtos disponíveis, deve ser exibida uma mensagem informando "Nenhum produto encontrado".
- A listagem deve ser acessível tanto por visitantes quanto por usuários logados.

Critérios de Priorização:

Prioridade: Alta (*Must Have*)
Justificativa: a listagem de produtos é essencial para o funcionamento do *e-commerce*, sendo a principal forma de exibir o catálogo da loja.

3.1.3. Página do produto

ID: US03

Como cliente, **quero** visualizar os detalhes de um produto selecionado, **para** compreender suas características e decidir se desejo comprá-lo.

Critérios de Aceitação:

- O sistema deve exibir uma página individual para cada produto.
- A página deve conter: imagem ampliada, nome, descrição, preço, disponibilidade em estoque e opção de quantidade.
- Deve haver um botão para adicionar o produto ao carrinho.
- Deve haver um botão para adicionar o produto aos favoritos.
- Caso o produto esteja esgotado, o botão de compra deve estar desativado e uma mensagem de "Produto indisponível" deve ser exibida.
- O sistema deve permitir retornar à listagem de produtos facilmente (ex: botão "Voltar" ou "Continuar comprando").
- As informações devem ser consistentes com o banco de dados, atualizando automaticamente caso o preço ou estoque mudem.

Critérios de Priorização:

Prioridade: Alta (*Must Have*)
Justificativa: A página do produto é fundamental para apresentar os detalhes que influenciam a decisão de compra do usuário.

3.1.4. Favoritos

ID: US04

Como cliente, **quero** adicionar produtos à minha lista de favoritos, **para** poder salvá-los e acessá-los facilmente em outro momento.

Critérios de Aceitação:

- O sistema deve permitir adicionar ou remover produtos dos favoritos por meio de um ícone (ex: coração).
- O usuário deve poder acessar sua lista de favoritos a partir do menu principal.
- A lista deve exibir imagem, nome, preço e link para a página do produto.
- Somente usuários logados podem salvar produtos nos favoritos.
- Caso o usuário não esteja logado e tente favoritar algo, o sistema deve solicitar login.
- Os produtos favoritados devem permanecer salvos na conta do usuário (armazenados no banco de dados).
- O sistema deve exibir uma mensagem de confirmação ao adicionar ou remover um item dos favoritos.

Critérios de Priorização:

Prioridade: Média (*Should Have*)
Justificativa: A funcionalidade melhora a experiência do usuário, mas não é essencial para o funcionamento básico do marketplace.

3.1.5. Carrinho

ID: US05

Como cliente, **quero** adicionar produtos ao carrinho, **para** visualizar o total da compra e finalizar o pedido de forma prática.

Critérios de Aceitação:

- O sistema deve permitir adicionar produtos ao carrinho diretamente da listagem ou da página do produto.
- O carrinho deve exibir imagem, nome, quantidade, preço unitário e subtotal de cada item.
- O usuário deve poder alterar a quantidade ou remover itens do carrinho.

- O sistema deve atualizar automaticamente o valor total sempre que houver alterações.
- Deve haver um botão "Finalizar compra" que direcione para a etapa de pagamento.
- Caso o carrinho esteja vazio, o sistema deve exibir uma mensagem como "Sua sacola está vazia" e sugerir retornar à loja.
- O carrinho deve ser salvo na conta do usuário logado ou armazenado temporariamente no navegador (para visitantes).
- O sistema deve garantir que produtos fora de estoque não possam ser adicionados.

Critérios de Priorização:

Prioridade: Alta (*Must Have*)
Justificativa: O carrinho é essencial para a experiência de compra e conclusão dos pedidos dentro do *e-commerce*.

3.1.6. Finalizar compra

ID: US06

Como cliente, **quero** finalizar minha compra escolhendo a forma de pagamento e o endereço de entrega, **para** concluir o pedido com segurança e praticidade.

Critérios de Aceitação:

- O sistema deve permitir visualizar um resumo do pedido, com produtos, quantidades e valor total.
- O usuário deve poder informar ou selecionar um endereço de entrega.
- Devem estar disponíveis métodos de pagamento como Pix, cartão de crédito e boleto.
- O sistema deve validar os dados de pagamento antes de concluir a compra.
- Após o pagamento, o sistema deve gerar um número de pedido e exibir uma mensagem de confirmação.
- O pedido deve ser registrado no banco de dados e associado à conta do usuário.
- O usuário deve receber um e-mail de confirmação com os detalhes da compra.
- Caso o pagamento falhe, o sistema deve exibir mensagem de erro clara e permitir tentar novamente.

Critérios de Priorização:

Prioridade: Alta (*Must Have*)
Justificativa: A finalização de compra é o núcleo do *e-commerce*, sendo indispensável para a concretização das vendas.

3.1.7. Atendimento/Contato

ID: US07

Como cliente, **quero** poder entrar em contato com a loja, **para** tirar dúvidas, solicitar informações sobre produtos ou resolver problemas relacionados à compra.

Critérios de Aceitação:

- O sistema deve oferecer uma página ou seção de contato acessível pelo menu principal.
- O usuário deve poder enviar uma mensagem preenchendo nome, e-mail e descrição da dúvida.
- O sistema deve confirmar o envio da mensagem com uma notificação visível ("Mensagem enviada com sucesso").
- As mensagens devem ser armazenadas no banco de dados e enviadas ao e-mail da loja.
- Deve haver link direto para o WhatsApp ou outro canal de atendimento rápido.
- O sistema deve validar campos obrigatórios (nome, e-mail e mensagem) antes do envio.
- Em caso de erro no envio, o sistema deve exibir uma mensagem explicando o problema.
- O administrador deve poder visualizar e responder mensagens pelo painel da loja (em versões futuras).

Critérios de Priorização:

Prioridade: Média (*Should Have*)
Justificativa: O canal de contato melhora o relacionamento com o cliente e o suporte, mas não impede o funcionamento principal do marketplace.

3.1.8. Administração (para a dona da empresa)

ID: US08

Como administrador da empresa, **quero** cadastrar, editar e remover produtos do sistema, **para** manter o catálogo sempre atualizado e organizado.

Critérios de Aceitação:

- O sistema deve permitir cadastrar novos produtos, informando nome, descrição, preço, categoria, quantidade em estoque e imagem.
- Deve ser possível editar informações de produtos já cadastrados.
- O administrador deve poder excluir produtos do catálogo, com confirmação antes da exclusão.
- Todos os campos obrigatórios devem ser validados antes de salvar (ex: nome, preço, categoria).
- O sistema deve armazenar as imagens dos produtos de forma segura e associá-las corretamente aos registros.
- As alterações devem refletir imediatamente na listagem de produtos exibida aos clientes.

- Apenas usuários com perfil de administrador devem ter acesso a essa funcionalidade.
- O sistema deve exibir mensagens de sucesso ou erro após cada operação (cadastro, edição ou exclusão).

Critérios de Priorização:

Prioridade: Alta (*Must Have*)
Justificativa: Essencial para manter o catálogo de produtos atualizado e garantir a operação contínua da loja online.

3.2. Casos de Uso

Um caso de uso especifica o comportamento de um sistema, ou de parte dele, e consiste na descrição de um conjunto de sequências de ações [(BOOCH et al., 2005[3])]. Essas sequências incluem as funções principais que o sistema deve executar, bem como suas variações, de forma a produzir um resultado que seja observável pelo ator que interage com o sistema [MENOLLI 2024[1]].

Diferentemente das histórias de usuário, que adotam uma abordagem ágil voltada ao valor percebido pelo usuário, o caso de uso possui uma abordagem mais tradicional, focada na especificação detalhada do sistema. Ele descreve como o sistema se comporta diante do usuário, e não apenas o que o usuário deseja ou qual o benefício esperado.

Um caso de uso é, portanto, mais estruturado, detalhado e técnico, contendo elementos como identificador, atores envolvidos, fluxos principais e alternativos, pré-condições e pós-condições.

Para esta atividade, os casos de uso serão elaborados com base nas histórias de usuário previamente definidas, servindo como detalhamento técnico das funcionalidades mais importantes do sistema. Essa abordagem permite conectar a perspectiva do usuário com a implementação prática do sistema, garantindo que os requisitos levantados sejam claros, rastreáveis e executáveis.

O caso de uso deve **conter**:

1. Identificador
2. Nome do Caso de Uso
3. Atores Envolvidos
4. Breve Descrição
5. Fluxo Principal de Eventos
6. Fluxos Alternativos
7. Pré-condições
8. Pós-condições

ID: UC01

Nome: Cadastro de Usuário

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo de criação de uma nova conta de usuário no sistema de e-commerce. O objetivo é permitir que novos usuários se cadastrem para utilizar as funcionalidades restritas do sistema.

Pré-condições:
- O sistema deve estar conectado ao banco de dados.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa a página inicial e seleciona a opção "Cadastrar-se". 2. O sistema exibe o formulário de cadastro com campos obrigatórios (nome, e-mail e senha). 3. O usuário preenche os campos solicitados. 4. O usuário confirma os dados e envia o formulário. 5. O sistema valida as informações fornecidas. 6. O sistema cria o registro do novo usuário. 7. O sistema realiza o login automático e redireciona o usuário para a página inicial autenticada.	**2.a Erro ao carregar formulário** O sistema exibe mensagem de falha e sugere tentar novamente mais tarde. **3.a Campos obrigatórios não preenchidos** O sistema alerta o usuário para completar todos os campos antes de prosseguir. **3.b Formato de e-mail inválido** O sistema solicita que o usuário insira um e-mail válido. **5.a E-mail já cadastrado** O sistema informa que o endereço de e-mail já está em uso e solicita outro. **6.a Erro de conexão** O sistema exibe mensagem de falha e orienta o usuário a tentar novamente mais tarde. **6.b Falha ao registrar usuário** O sistema notifica que ocorreu um erro interno e cancela o processo de cadastro.

Pós-condições:
- Uma nova conta de usuário é criada e armazenada no banco de dados.
- O usuário é autenticado automaticamente e redirecionado para a área principal do sistema.

ID: UC02

Nome: Login de Usuário

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo de autenticação de um usuário previamente registrado no sistema de e-commerce. O objetivo é permitir que o usuário acesse sua conta e utilize funcionalidades restritas, como favoritos, carrinho e histórico de pedidos.

Pré-condições:
- O sistema deve estar conectado ao banco de dados.
- O usuário deve possuir uma conta previamente registrada.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa a página inicial e seleciona a opção “Entrar”. 2. O sistema exibe a tela de login com campos de e-mail e senha. 3. O usuário insere suas credenciais e confirma o envio. 4. O sistema valida as informações fornecidas. 5. Se as credenciais forem válidas, o sistema autentica o usuário. 6. O sistema redireciona o usuário para a página inicial autenticada.	**2.a Erro ao carregar tela de login** O sistema exibe uma mensagem de falha e orienta o usuário a tentar novamente mais tarde.
	3.a Campos obrigatórios não preenchidos O sistema solicita o preenchimento de e-mail e senha antes do envio.
	4.a Credenciais inválidas O sistema exibe uma mensagem de erro e solicita nova tentativa. **4.b E-mail não cadastrado** O sistema informa que o e-mail não existe e sugere o cadastro de uma nova conta. **4.c Erro de conexão** O sistema exibe mensagem de falha e orienta o usuário a tentar novamente mais tarde.
	5.a Conta bloqueada O sistema notifica o usuário sobre a restrição e sugere contato com o suporte.

Pós-condições:
- O usuário é autenticado com sucesso e tem acesso às funcionalidades exclusivas do sistema.

ID: UC03

Nome: Listagem de Produtos

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo de exibição dos produtos disponíveis no e-commerce. O usuário pode visualizar os itens, aplicar filtros, navegar por categorias e acessar detalhes de um produto específico.

Pré-condições:
- O sistema deve possuir produtos cadastrados.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa a página inicial do e-commerce. 2. O sistema exibe uma lista com os produtos disponíveis. 3. O usuário pode rolar a página para visualizar mais itens. 4. O usuário pode utilizar a barra de pesquisa para localizar produtos específicos. 5. O usuário pode aplicar filtros ou selecionar categorias (ex: tecidos, botões, linhas, cordões etc.). 6. O sistema atualiza a listagem conforme a pesquisa ou filtros aplicados. 7. O usuário seleciona um produto para visualizar seus detalhes.	**2.a Nenhum produto cadastrado** O sistema exibe uma mensagem informando que não há produtos disponíveis no momento. **5.a Erro na aplicação de filtros** O sistema mantém a listagem original e exibe mensagem de falha na filtragem. **7.a Erro ao carregar detalhes do produto:** O sistema exibe mensagem de erro e solicita nova tentativa.

Pós-condições:
- O usuário pode acessar a página de detalhes de um produto selecionado.
- O sistema mantém registro das preferências de visualização ou filtros aplicados (se o usuário estiver autenticado).

ID: UC04

Nome: Página do Produto

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo de exibição detalhada de um produto selecionado pelo usuário no e-commerce. O objetivo é apresentar informações completas sobre o item, como imagem, descrição, preço, disponibilidade e opções de compra, de forma a auxiliar o cliente em sua decisão de aquisição.

Pré-condições:
- O sistema deve estar conectado ao banco de dados.
- O produto selecionado deve existir no catálogo.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário seleciona um produto na listagem. 2. O sistema carrega a página individual do produto. 3. O sistema exibe as seguintes informações: imagem ampliada, nome, descrição, preço, disponibilidade em estoque e campo para selecionar a quantidade desejada. 4. O usuário pode adicionar o produto ao carrinho clicando no botão "Adicionar ao Carrinho". 5. O usuário pode favoritar o produto clicando no botão "Adicionar aos Favoritos". 6. O sistema confirma a ação com uma mensagem visual (ex.: "Produto adicionado com sucesso"). 7. O usuário pode retornar à listagem de produtos utilizando o botão "Voltar" ou "Continuar Comprando". 8. O sistema mantém as informações do produto atualizadas com base nos dados do banco (preço e estoque).	**2.a Erro ao carregar a página** O sistema exibe uma mensagem de falha e orienta o usuário a tentar novamente mais tarde. **3.a Produto indisponível** O botão de compra é desativado e o sistema exibe a mensagem "Produto indisponível". **4.a Falha ao adicionar ao carrinho** O sistema informa que ocorreu um erro e solicita nova tentativa. **5.a Falha ao adicionar aos favoritos** O sistema exibe mensagem de erro e mantém o estado anterior. **8.a Atualização de preço ou estoque** O sistema atualiza automaticamente as informações exibidas em tempo real.

Pós-condições:
- As informações do produto são exibidas corretamente.
- O produto pode ser adicionado ao carrinho ou aos favoritos conforme a disponibilidade.

ID: UC05

Nome: Favoritos

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo em que o usuário pode adicionar, visualizar e gerenciar produtos na lista de favoritos. Essa funcionalidade permite que o usuário salve itens de interesse para acessá-los posteriormente, facilitando futuras compras ou comparações.

Pré-condições:

- O usuário deve estar autenticado no sistema para adicionar ou remover produtos dos favoritos.
- O produto deve estar cadastrado no sistema.
- O sistema deve estar conectado ao banco de dados.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa a página de um produto. 2. O sistema exibe o botão "Adicionar aos Favoritos". 3. O usuário clica no botão para favoritar o produto. 4. O sistema verifica se o usuário está autenticado. 5. O sistema registra o produto como favorito e confirma a ação. 6. O usuário pode acessar a lista de favoritos a partir do menu principal. 7. O sistema exibe todos os produtos favoritados com imagem, nome e preço. 8. O usuário pode remover produtos da lista, se desejar.	**3.a Usuário não autenticado** O sistema solicita login antes de permitir adicionar o produto aos favoritos. **5.a Erro ao salvar o produto** O sistema exibe uma mensagem de falha e solicita nova tentativa. **6.a Lista de favoritos vazia** O sistema exibe a mensagem "Nenhum produto favoritado ainda". **8.a Erro ao remover produto** O sistema exibe mensagem de erro e mantém o item na lista.

Pós-condições:

- O produto é adicionado ou removido da lista de favoritos conforme a ação do usuário.
- O sistema mantém o registro atualizado dos produtos favoritados, vinculado à conta do usuário.

ID: UC06

Nome: Carrinho de Compras

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo de gerenciamento dos produtos adicionados ao carrinho de compras. O usuário pode visualizar os itens selecionados, alterar quantidades, remover produtos e prosseguir para o fechamento do pedido.

Pré-condições:
- O usuário deve estar logado no sistema
- O usuário deve ter acessado o sistema e adicionado ao menos um produto ao carrinho.
- O sistema deve estar conectado ao banco de dados.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa o carrinho de compras a partir do menu ou após adicionar um produto. 2. O sistema exibe todos os produtos adicionados, com nome, imagem, quantidade, preço unitário e subtotal. 3. O usuário pode alterar a quantidade de um produto. 4. O sistema recalcula automaticamente o valor total da compra. 5. O usuário pode remover um item do carrinho. 6. O sistema atualiza a lista de produtos e o total. 7. O usuário clica em "Finalizar Compra". 8. O sistema redireciona o usuário para a página de **Finalização da Compra**.	**2.a Carrinho vazio** O sistema exibe a mensagem "Seu carrinho está vazio" e sugere voltar à listagem de produtos. **3.a Quantidade inválida** O sistema exibe mensagem informando que o valor inserido não é permitido. **5.a Falha ao remover produto** O sistema exibe mensagem de erro e mantém o item no carrinho. **7.a Usuário não autenticado** O sistema solicita login antes de permitir o fechamento da compra. **8.a Erro de conexão** O sistema exibe mensagem de falha e orienta o usuário a tentar novamente mais tarde.

Pós-condições:
- O carrinho é atualizado conforme as ações do usuário.
- O sistema mantém o estado do carrinho até a finalização ou o *logout* do usuário.

ID: UC7

Nome: Finalização da Compra

Ator: Usuário

Breve Descrição: Este caso de uso descreve o processo de finalização de compra realizado pelo cliente após a seleção dos produtos no carrinho. O usuário escolhe o endereço de entrega e a forma de pagamento, confirma o pedido e conclui a compra de forma segura e prática. O sistema registra o pedido, gera um número de confirmação e envia um e-mail com os detalhes ao comprador.

Pré-condições:
- O usuário deve estar autenticado no sistema.
- O carrinho deve conter ao menos um produto.
- O sistema deve estar conectado ao serviço de pagamento e ao banco de dados.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa sua cesta de compra. 2. O sistema mostra as descrições, quantidade e o preço de cada produto. 3. O sistema calcula o valor total de cada produto. 4. O cliente seleciona ”finalizar pedido”. 5. O cliente fornece seu nome e endereço. 6. Se o cliente fornece apenas o CEP, o sistema coloca automaticamente a cidade e o estado. 7. O sistema calcula o valor total do pedido. 8. O cliente seleciona o método de pagamento 9. O cliente fornece as informações sobre o cartão. 10. O cliente submete os dados ao sistema. 11. O sistema verifica as informações fornecidas e marca o pedido como ”pendente”, e o número de pedido (NP) é dado ao cliente.	**3.a Produto em Promoção** 1. (Ponto de extensão – Calcular Desconto de Produto em Promoção) 2. Retorna ao Fluxo Principal no passo 4.
	5.a Endereço Inexistente 1. O sistema emite uma mensagem que o endereço é inexistente. 2. Retorna ao Fluxo Principal no passo 5.
	6.a CEP Inválido 1. O sistema emite uma mensagem que o endereço é inexistente. 2. Retorna ao Fluxo Principal no passo 6.
	7.a Cupom de Desconto 1. O sistema calcula o valor do desconto. 2. Retorna ao Fluxo Principal no passo 8.
	9.a Pagamento via boleto 1. O sistema abre uma nova aba com as informações sobre boleto 2. O sistema emite um boleto com todas as informações sobre o pagamento 3. O cliente fecha essa tela 4. Retorna ao Fluxo Principal no Passo 10.
	9.b Pagamento por depósito 1. O sistema abre uma nova aba com as informações de depósito 2. O cliente insere as informações do depósito 3. O cliente submete os dados ao sistema 4. Retorna ao Fluxo Principal no Passo 10.

Pós-condições:
- O pedido é registrado no banco de dados e associado ao usuário.
- O sistema envia a confirmação por e-mail.
- O status do pedido é atualizado para “Pago” ou “Em Processamento”, conforme o retorno da transação.

Ponto de Extensão: 3.a Produto em Promoção - Calcular Desconto de Produto em Promoção

ID: UC8

Nome: Atendimento e Contato com o Suporte

Atores: Usuário, Equipe de Suporte

Breve Descrição: Este caso de uso descreve o processo pelo qual o usuário entra em contato com a equipe de atendimento do *e-commerce* para esclarecer dúvidas, relatar problemas ou solicitar informações adicionais. O sistema disponibiliza um formulário de contato e, opcionalmente, canais complementares como e-mail ou chat.

Pré-condições:
- O sistema deve estar conectado ao banco de dados.

Fluxo Principal	**Fluxos Alternativos**
Fluxo Principal (caminho básico): 1. O usuário acessa a página "Atendimento/Contato" através do menu principal ou rodapé do site. 2. O sistema exibe o formulário de contato, contendo campos como nome, e-mail, assunto e mensagem. 3. O usuário preenche os campos obrigatórios e envia a mensagem. 4. O sistema valida os dados inseridos (ex.: e-mail válido, mensagem não vazia). 5. O sistema registra a solicitação no banco de dados e encaminha o conteúdo para o setor de suporte. 6. O sistema exibe uma mensagem de confirmação informando que a solicitação foi enviada com sucesso. 7. A equipe de suporte recebe a solicitação e entra em contato com o usuário via e-mail cadastrado.	**2.a Erro ao carregar o formulário** O sistema exibe uma mensagem de falha e orienta o usuário a tentar novamente. **3.a Campos obrigatórios não preenchidos** O sistema informa quais campos estão faltando e solicita correção. **4.a E-mail inválido** O sistema exibe mensagem de erro solicitando um endereço de e-mail válido. A **5.a Falha no envio** O sistema exibe mensagem de erro e orienta o usuário a tentar novamente mais tarde. **7.a Falha de comunicação com o suporte** O sistema armazena a solicitação e informa que o atendimento será realizado assim que possível.

Pós-condições:
- A solicitação do usuário é registrada no sistema e encaminhada à equipe de suporte.
- O usuário recebe uma confirmação visual e, posteriormente, um retorno via e-mail.

<table>
<tr><td colspan="2">ID: UC09</td></tr>
<tr><td colspan="2">Nome: Administração (para a dona da loja)</td></tr>
<tr><td colspan="2">Ator: Administrador</td></tr>
<tr><td colspan="2">Breve Descrição: Permite que o administrador tenha acesso às funcionalidades administrativas, como o gerenciamento de produtos, controle de estoque, acompanhamento de pedidos e visualização de mensagens recebidas no atendimento.</td></tr>
<tr><td colspan="2">Pré-condições:
• O administrador deve estar autenticado no sistema.</td></tr>
<tr><td>Fluxo Principal</td><td>Fluxos Alternativos</td></tr>
<tr><td rowspan="2">1. O administrador acessa o painel administrativo.
2. O sistema exibe as opções disponíveis: Gerenciar Produtos, Gerenciar Pedidos, Mensagens de Clientes e Estoque.
3. O administrador seleciona uma das opções.
4. O sistema exibe a interface correspondente.
5. O administrador realiza as ações desejadas (adicionar, editar, excluir, visualizar).
6. O sistema confirma a operação e atualiza as informações.</td><td>3.a Nenhuma opção selecionada
O sistema mantém a tela principal de administração.</td></tr>
<tr><td>5.a Erro ao salvar alterações
O sistema exibe uma mensagem de erro e solicita que a operação seja tentada novamente.

5.b Conexão perdida
O sistema alerta sobre a falha de conexão e oferece a opção de tentar novamente.</td></tr>
<tr><td colspan="2">Pós-condições:
• Alterações realizadas pelo administrador (como inclusão, edição ou exclusão de produtos) são salvas no banco de dados.</td></tr>
</table>

3.3. Diagrama de Classe

O diagrama de classes deriva do modelo de classes e tem como objetivo facilitar a leitura e compreensão de como o sistema será estruturado tecnicamente, mostrando suas classes, atributos, métodos e relacionamentos. Nesta etapa, buscamos identificar e entender quais classes são necessárias e como elas interagem entre si para implementar o sistema proposto [1].

Para a criação do modelo de análise, é essencial considerar três aspectos principais:

- **Quais classes existem no sistema?**
  São as entidades principais que representam os objetos do mundo real ou conceitos do domínio.
- **Qual é a responsabilidade de cada classe?**
  Cada classe deve ter um papel bem definido, descrevendo o que ela faz e quais dados manipula.
- **Cada classe funciona sozinha ou precisa de outras para cumprir sua função?**
  Essa análise determina os relacionamentos necessários, como associações, composições e heranças.

(Menolli, 2024 [1])

3.3.1. Relação com os Casos de Uso

O diagrama de classes foi elaborado a partir das histórias de usuário e dos casos de uso identificados anteriormente, garantindo que a estrutura do sistema atenda aos requisitos funcionais levantados. Por exemplo:

- O caso de uso **"Cadastro e Login de Usuário"** originou a classe `Usuário`, responsável pelos dados de acesso e identificação.
- O caso de uso **"Gerenciamento de Produtos"** gerou as classes `Produto`, `Categoria` e `Administrador`, mostrando o controle de estoque e categorias de itens.
- O caso de uso **"Atendimento/Contato"** deu origem à classe `Mensagem`, permitindo a comunicação entre o usuário e o administrador.
- O caso de uso **"Finalizar Compra"** definiu as classes `Carrinho`, `ItemCarrinho`, `Pedido` e `Pagamento`, responsáveis por todo o fluxo de compra.

Dessa forma, o diagrama de classes reflete diretamente as interações e comportamentos descritos nos casos de uso, traduzindo a visão funcional em uma estrutura técnica orientada a objetos.

3.3.2. Relações entre classes no Diagrama do Projeto

O diagrama apresenta diferentes tipos de relacionamentos, cada um representando um tipo de dependência entre as classes:

- **Associação:** representa uma relação simples entre duas classes, em que uma utiliza a outra, mas sem dependência forte.
  Exemplo: `Usuário` está associado a `Endereço`, pois um usuário pode ter vários endereços.
- **Composição:** indica uma relação de dependência forte, onde o objeto "filho" não existe sem o "pai".
  Exemplo: `Carrinho` e `ItemCarrinho` — se o carrinho for excluído, seus itens também deixam de existir.
- **Herança (Generalização):** indica que uma classe filha herda atributos e comportamentos de uma classe pai.
  Exemplo: `Administrador` herda de `Usuário`, compartilhando características básicas, mas adicionando funções específicas como gerenciar produtos e responder mensagens.
- **Classe de Associação:** ocorre quando a relação entre duas classes possui atributos próprios e precisa ser modelada como uma classe independente.
  Exemplo: a classe `Favoritos` representa a associação entre `Usuário` e `Produto`, registrando quais produtos foram marcados como favoritos por cada usuário.

3.3.3. Conclusão

O diagrama de classes elaborado reflete de maneira organizada e coerente a estrutura lógica do sistema, derivada dos casos de uso e dos requisitos levantados. Ele mostra as

entidades essenciais, suas responsabilidades e como elas se conectam para garantir o funcionamento integrado das funcionalidades de cadastro, compras, pagamento, atendimento e administração da aplicação.

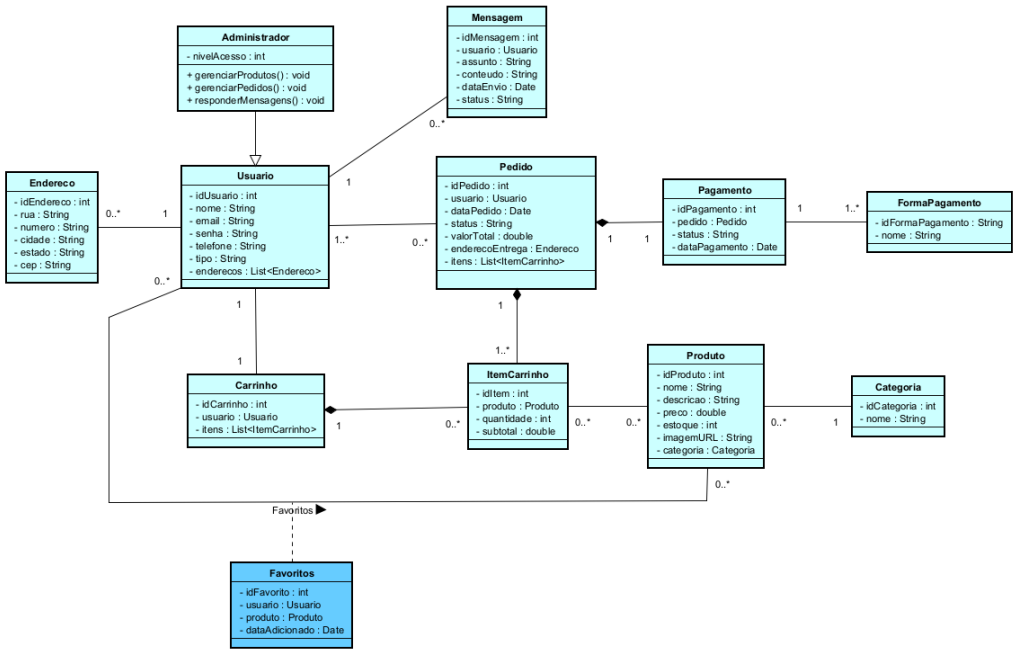


Figura 2. Diagrama de classe

3.4. Mapeamento Objeto Relacional

Como abordado no tópico anterior, o modelo de classes representa uma visão lógica do sistema, descrevendo as classes, seus atributos, métodos e relacionamentos. Esse modelo é voltado ao desenvolvimento orientado a objetos, sendo fundamental para compreender a estrutura interna do software e a forma como os componentes interagem entre si.

Por outro lado, o modelo entidade-relacionamento (MER) tem como foco apenas os dados e a estrutura do banco de dados. Nesse caso, o diagrama não representa classes e métodos, mas sim tabelas, colunas e relacionamentos entre elas. As classes do modelo lógico são, portanto, convertidas em tabelas, e os atributos das classes tornam-se campos (colunas) nessas tabelas. As associações, por sua vez, são traduzidas em chaves primárias (PK) e chaves estrangeiras (FK), de modo a preservar a integridade referencial.

O processo de conversão entre o modelo de classes e o modelo de banco de dados é denominado mapeamento objeto-relacional. Ele segue um conjunto de regras e boas práticas que garantem a correspondência adequada entre os dois modelos, conforme descrito a seguir:

- **Relação 1 para 1 (1:1):** Quando duas classes mantêm uma associação um-para-um, apenas uma das tabelas resultantes deve conter a chave estrangeira (FK) da outra. Essa FK deve possuir a restrição *UNIQUE*, garantindo que cada registro esteja vinculado a, no máximo, um registro da outra tabela. Exemplo: um cliente possui um único carrinho; nesse caso, a tabela Carrinho contém uma FK idCliente com restrição de unicidade.

- **Relação 1 para Muitos (1:N):** Nessa relação, uma entidade "pai" está associada a várias entidades "filhas". A tabela correspondente à entidade "filha" deve conter a FK que referencia a tabela "pai". Exemplo: um cliente pode ter vários pedidos, logo a tabela Pedido contém a FK idCliente.
- **Relação Muitos para Muitos (N:M):** Quando duas classes possuem uma relação bidirecional em que ambas podem se associar a múltiplos registros da outra, é criada uma tabela intermediária (também chamada de classe associativa). Essa tabela contém duas FKs — uma para cada entidade envolvida, e pode, opcionalmente, incluir atributos próprios. Exemplo: usuários e produtos possuem uma relação N:M através da tabela Favorito, que contém idUsuario e idProduto como FKs.
- **Herança:** Em casos de generalização/especialização, há três estratégias de mapeamento possíveis: (a) Tabela única (single table): todas as subclasses são armazenadas na mesma tabela, com colunas opcionais para atributos específicos; (b) Tabela por classe concreta: cada subclasse possui sua própria tabela independente; (c) Tabela por hierarquia: cada subclasse tem sua própria tabela, contendo apenas seus atributos específicos e uma FK para a tabela da superclasse.
- **Composição:** Na composição, uma classe "filha" não existe sem a "classe pai". No banco de dados, isso significa que a tabela da parte dependente contém uma FK obrigatória (*NOT NULL*) referenciando a tabela principal, e sua exclusão pode ser configurada em cascata (*ON DELETE CASCADE*).

Essas regras permitem que o modelo orientado a objetos seja traduzido de forma coerente para o modelo relacional, garantindo que as relações lógicas do software sejam mantidas de forma consistente e funcional no banco de dados.

A seguir, observe como o Mapeamento Objeto Relacional do projeto em questão se desenvolve:

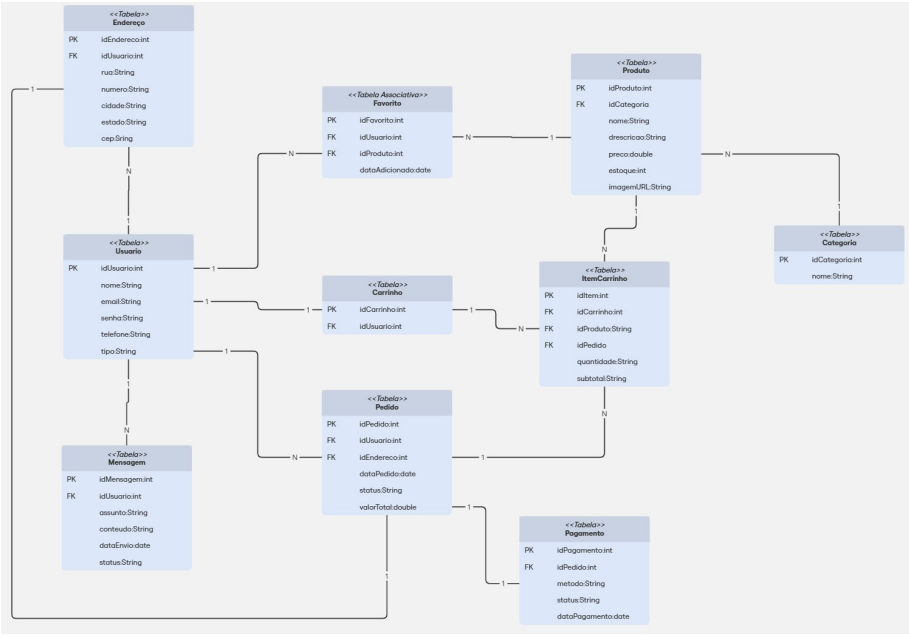


Figura 3. Mapeamento Objeto Relacional

3.5. Arquitetura da aplicação

A arquitetura consiste em um modelo de alto nível que auxilia no entendimento e análise do software a ser desenvolvido e ganhou destaque para representar soluções de software principalmente por (GARLAN e PERRY, 1995[4]):

- A abstração facilita a visualização e o entendimento de certas propriedades do software.
- A exploração cada vez maior de *frameworks* visando diminuir o esforço de construção de produtos por meio da integração de partes previamente desenvolvidas.

Além disso, a arquitetura atua como um meio de comunicação entre as partes interessadas no projeto, como desenvolvedores, analistas, gerentes e clientes. Ela permite alinhar expectativas e estabelecer uma base comum de entendimento sobre o funcionamento e as restrições do sistema (MENOLLI, 2024[1]).

Outro aspecto fundamental é que a arquitetura influencia diretamente as decisões de implementação, facilitando a definição das camadas e a escolha das tecnologias adequadas. A separação em camadas como Apresentação, Negócio e Persistência contribui para o desacoplamento e para a reutilização de código, favorecendo a evolução e a manutenção do sistema ao longo do tempo.

Por fim, a modelagem arquitetural permite representar o fluxo de informações entre os componentes e suas dependências. Essa representação é especialmente útil para identificar gargalos, riscos técnicos e oportunidades de otimização, assegurando que o sistema atenda aos requisitos funcionais e não funcionais estabelecidos durante o projeto.

A arquitetura da aplicação proposta adota um modelo em camadas, com o objetivo de organizar o sistema de forma modular e clara, garantindo separação de responsabilidades, manutenibilidade e facilidade de escalabilidade.

A Figura 4. apresenta o diagrama de arquitetura da solução. Nele, é possível observar o fluxo completo de comunicação entre o usuário, o front-end desenvolvido em Flutter, a camada de negócios em Dart e o banco de dados Firebase, que atua como camada de persistência NoSQL.

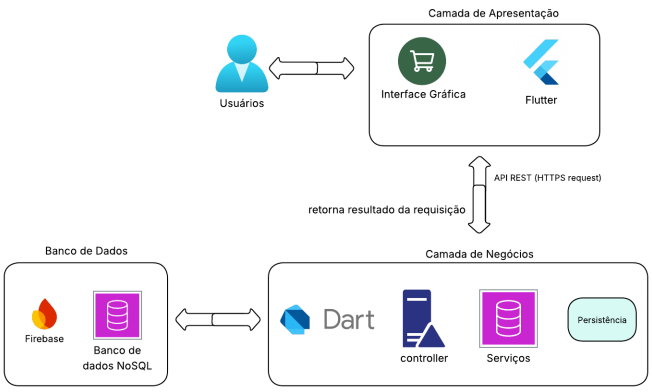


Figura 4. Arquitetura de aplicação

3.5.1. Usuário e Camada de Apresentação (Front-end)

A interação com o sistema inicia-se na Camada de Apresentação, onde os usuários acessam a interface gráfica construída com o framework Flutter. O Flutter foi escolhido por permitir o desenvolvimento multiplataforma (Android, iOS e Web) com alto desempenho e uma interface moderna e responsiva, facilitando a manutenção do código e a experiência do usuário.

Essa camada é responsável por:

- Capturar ações e dados inseridos pelo usuário (cadastro, login, pedidos etc.);
- Exibir informações retornadas da lógica de negócios;
- Enviar requisições HTTPS à camada de negócio, utilizando o padrão API REST.

3.5.2. Camada de Negócios

A Camada de Negócios implementa as regras de funcionamento do sistema e é desenvolvida em Dart, mesma linguagem utilizada no front-end, o que simplifica a integração entre as partes. Nessa camada estão os *Controllers* e *Services*, responsáveis por:

- Processar as requisições recebidas do front-end;
- Executar validações e regras do domínio (ex.: cálculo de total de pedidos, autenticação, manipulação de dados);
- Realizar a comunicação direta com o Firebase por meio de serviços de persistência.

Diferente de arquiteturas tradicionais que mantêm um backend dedicado, nesta solução a persistência é parcialmente tratada dentro da própria camada de negócio, pois o Firebase já fornece recursos prontos de backend, como autenticação, API de dados e hospedagem. Dessa forma, a aplicação envia requisições diretamente à plataforma do Firebase, sem necessidade de um servidor intermediário.

3.5.3. Camada de Persistência e Banco de Dados

A camada de persistência é representada pelo Firebase, que funciona como Backend as a Service (BaaS). Ele oferece uma infraestrutura pronta para armazenar, autenticar e sincronizar dados em tempo real com o aplicativo, eliminando a necessidade de construir e manter um servidor próprio.

O banco de dados utilizado é o Firebase Firestore, um banco NoSQL que armazena dados em formato de documentos e coleções, proporcionando maior flexibilidade de estrutura e escalabilidade.

As principais vantagens dessa abordagem incluem:

- Sincronização em tempo real entre os dispositivos dos usuários;
- Redução da complexidade de gerenciamento de servidores;
- Integração nativa com o Flutter e o SDK Dart;
- Segurança e autenticação integradas via Firebase Authentication.

3.5.4. Comunicação entre as Camadas

O fluxo de comunicação ocorre da seguinte forma:

- O usuário interage com a interface Flutter (Camada de Apresentação);
- As ações são convertidas em requisições REST enviadas à Camada de Negócio;
- Os controllers e serviços Dart processam os dados e realizam operações sobre o Firebase;
- O Firebase executa as ações de leitura, escrita ou autenticação e retorna os resultados;
- A resposta é enviada de volta ao front-end, que atualiza a interface do usuário.

Essa arquitetura em camadas permite uma alta coesão interna e baixo acoplamento entre os componentes, possibilitando que cada parte evolua de forma independente.

- A abstração facilita a visualização e o entendimento de certas propriedades do software.
- A exploração cada vez maior de frameworks visando diminuir o esforço de construção de produtos por meio da integração de partes previamente desenvolvidas.

3.6. Considerações Finais

O desenvolvimento deste trabalho teve como principal objetivo promover a integração entre a comunidade e a instituição de ensino, proporcionando uma contribuição significativa para os cidadãos e aproximando o conhecimento acadêmico das necessidades reais da sociedade.

A realização do projeto permitiu aplicar, na prática, os conceitos estudados ao longo da disciplina de Engenharia de Software II, evidenciando a importância dessa área no planejamento e construção de sistemas eficazes. Por meio da definição dos requisitos, modelagem e representação arquitetural, foi possível compreender com maior clareza as etapas que envolvem o desenvolvimento de uma aplicação e os desafios inerentes à sua execução.

Além disso, o processo de documentação e modelagem demonstrou o valor do planejamento prévio antes da implementação. Ao representar graficamente a estrutura e o funcionamento do sistema, pôde-se antecipar dificuldades, propor soluções e organizar de maneira eficiente o fluxo de desenvolvimento. Dessa forma, o trabalho reforça a relevância da engenharia de software como um instrumento essencial para a criação de soluções tecnológicas consistentes, bem estruturadas e alinhadas às demandas do contexto em que se inserem.

Referências

[1] MENOLLI, André. **Engenharia de Software: Do Requisito ao Projeto**. 1. ed. 2024.

[2] CRUZ, Carolina David; BADARÓ, Laura Moreira. **Estudo de aplicações de user story por profissionais da área de tecnologia da informação**. Belo Horizonte: PUC Minas, Instituto Politécnico, Departamento de Engenharia de Produção, [s.d.]. Disponível em: `http://bib.pucminas.br:8080/pergamumweb/vinculos/0000b4/0000b47a.pdf`. Acesso em: 2 nov. 2025.

[3] Booch, Grady; Rumbaugh, James; Jacobson, Ivar. *The Unified Modeling Language User Guide*. 2nd ed. Reading, MA: Addison-Wesley, 1999.

[4] GARLAN, David. Software Architecture. In: MARCINIAK, John J. (Ed.). Encyclopedia of Software Engineering. Hoboken: Wiley, 2002. Disponível em: https://doi.org/10.1002/0471028959.sof012. Acesso em: 2 nov. 2025.

Anexo

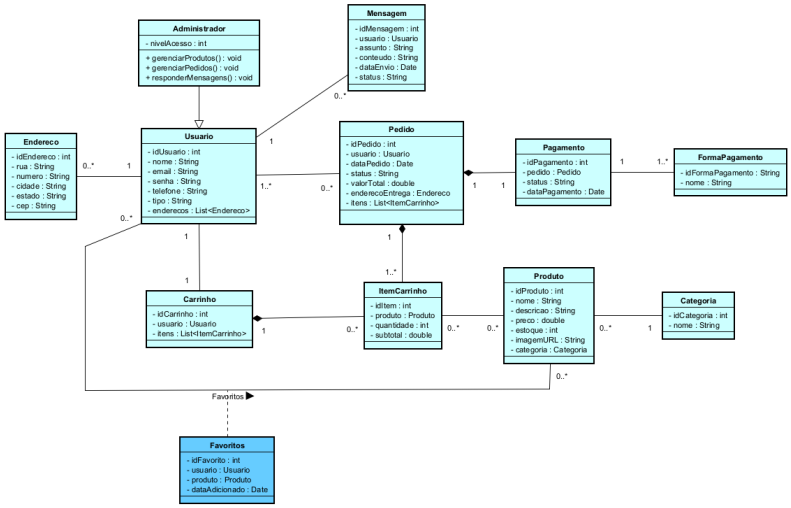


Figura 5. Diagrama de classe

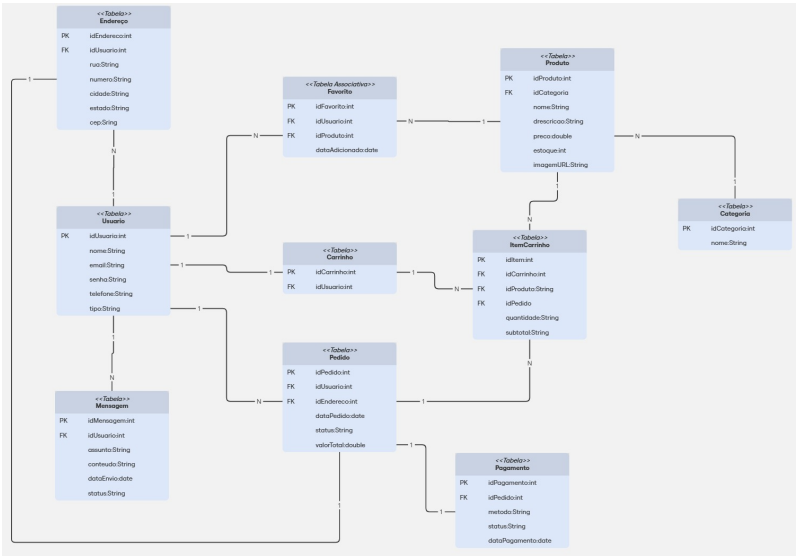


Figura 6. Mapeamento Objeto Relacional

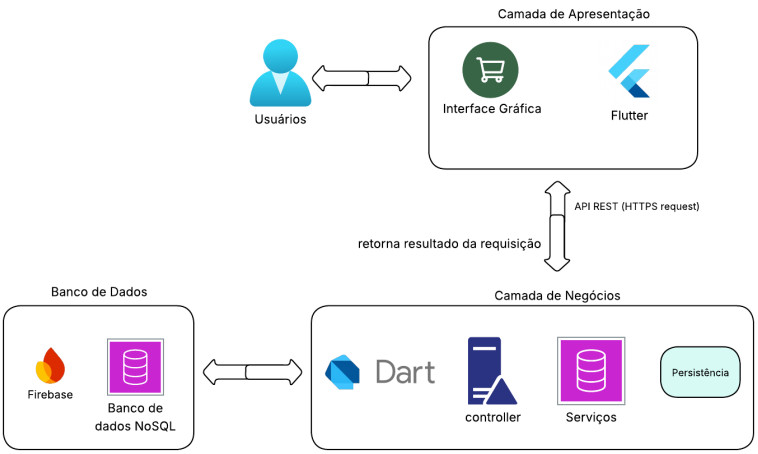


Figura 7. Arquitetura de aplicação